

NAME	CLASS	SPEAKER	DATE & TIME
Chris Miquel	Programación	Pichardo	

Title

Algoritmos, Diagramas de flujo

Keyword

Topic

Algoritmo

Fases de resolución

un problema es una situación que requiere una solución mediante el procesamiento de información.

un algoritmo: es una secuencia ordenada, finita y precisa de pasos para resolver un problema específico.

Questions

¿qué condiciones debe cumplir un algoritmo eficiente?

¿cuál es la diferencia entre el análisis del problema y el diseño de la solución?

características de un algoritmo
Preciso: debe indicar claramente el orden de realización de cada paso.

Definido: si se sigue dos veces con los mismos datos de entrada, debe dar lo mismo resultado.

Finito: debe terminar en algún momento.

Entradas/Salidas: por lo menos una entrada y al menos una salida.

Etapas en la resolución de problemas.

Análisis: comprender los requisitos, definir entradas y salidas y restricciones.

Summary:

Los algoritmos constituyen la base conceptual antes de escribir código formal. Para resolver un problema eficazmente es necesario analizarlo detenidamente, estructurar los pasos de forma clara y verificar la validez lógica del algoritmo planteado.

NAME	CLASS	SPEAKER	DATE & TIME
Elvis Miquel T.	Programación	Pichardo	

Title

Diagrama de flujo y sus reglas.

Keyword

Inicio/Fin

Proceso

Decision

Entrada/Salida

Topic

concepto de diagrama de flujo
Es la representación gráfica de un algoritmo mediante símbolos estandarizados unidos por líneas de flujo.

Símbolos principales.

Óvalo / óvalo alargado: inicio y fin del algoritmo.

Paralelogramo: operaciones de entrada y salida.

Rectángulo: procesos y asignación de datos.

Rombo: decisión / condición (evalúa expresiones verdaderas / falsas).

Círculo: conectar dentro de la misma página.

Reglas para la construcción de Diagrama de flujo:

Se deben escribir de arriba hacia abajo y de izquierda a derecha (Top-Down).

Questions

¿por qué es fundamental seguir reglas estandarizadas al graficar un flujo?

Causa vertidos, abre el diagrama de flujo pronto al código directo.

Summary:

Los diagramas de flujo preparan una representación visual estandarizada de la lógica de un algoritmo. Seguir las reglas formales de construcción garantiza que el diseño sea legible y fácil de trasladar a un lenguaje de programación como C.

NAME	CLASS	SPEAKER	DATE & TIME
Elvis Miqueos	programación	Pichardo	

Title

Tipos de datos, identificadores y constantes.

Keyword

Tipos de datos
Primitivos
int, float,
char, double

Topic

Tipos de datos Basicos en C:

char: caracteres individuales (1 byte)

int: Numeros enteros (2 a 4 bytes segun arquitectura)

float: Numeros en coma flotante de precision simple

double: Numeros reales de doble precision

Identificadores

Nombres asignados a variable, funciones y constantes

Questions

¿Como afecta la eleccion del tipo de dato a la memoria?

Reglas: Deben comenzar con una letra (a-z, A-Z) o guion bajo. No pueden contener espacios ni caracteres especiales sensibles a mayuscula y minuscula (case-sensitive)

¿cuales son los palabras reservadas que no pueden ser usadas como identificadores?

Constantes

Valores fijos que no cambian durante la ejecucion del programa.

Forma de declaracion

Directiva del preprocesador
Calificador de tipo

Summary:

El manejo adecuado de los tipos de datos y la correcta definicion de identificadores y constantes asegura la precision en el almacenamiento de memoria y mantiene la claridad organizativa del codigo en C.

NAME	CLASS	SPEAKER	DATE & TIME
Elvis Miquel	Program	P. Richard	

Title

Variables y operadores Aritméticos en C

Keyword
Declaración,
inicialización,
ambito

Topic

Variables en C:

Espacios reservados en la memoria RAM cuya información puede modificarse.

Sintaxis: tipo de datos nombre variable = Valor inicial;

Ejemplo: `int edad = 20;`

operadores aritméticos Básicos

(suma) (Resta) (multiplicación)

(División): si los operadores son enteros, el resultado es la parte entera (modulo). devuelve el residuo de una división entre enteros

operadores aritméticos simplificados
 $x += y$ equivale a $x = x + y$

Questions

¿Cuál es la diferencia entre la división entera y la división flotante en C?

¿Cuál uso práctico tiene el operador modulo?

Summary: Las Variables permiten almacenar datos dinámicos en el programa. A través de los operadores aritméticos básicos y simplificados se realizan los cálculos numéricos fundamentales dentro de C.

NAME	CLASS	SPEAKER	DATE & TIME
Elvis Miquel	Program	Pichardo	

Title

Operadores de incremento, decremento.

Keyword
Incremento/
Decremento

Topic operadores de incremento y
Decremento.

++ (Aumenta en 1 el valor del operador).
-- (Disminuye en 1 el valor operando).
Prefijo (++a / --a): se modifica el
valor antes de evaluar la expresión.
Sufijo (a++ / a--): se evalúa la expresión
con el valor actual y luego se
incrementa / decrementa.

Operadores Relacionales

Se utiliza para comparar dos valores:

> (mayor que), < (menor que)
>= (mayor o igual que), <= (menor o
igual que)
== (igual a), != (diferente de)

Evaluación de resultados relacionales

En C no existe un tipo primitivo
booleano tradicional en estándares
antiguos; el resultado de una comparación
es 1 para Verdadero (True) y 0 para
Falso (False)

Questions

¿Qué valor
numérico
devuelven
los compara-
ciones lógicas
en C?

Summary:

Los operadores de incremento
y decremento facilitan el manejo iterativo de
variables. Los operadores relacionales permiten
comparar valores y son fundamentales para
el control de flujo condicional.

NAME	CLASS	SPEAKER	DATE & TIME
Elvis Miqueos	Programación	Pichardo	

Title

Expresiones lógicas, operador coma y prioridad

Keyword

(and)

(or)

(Not)

Topic

operadores lógicos

&& (AND) Verdadero solo si ambas condiciones son verdaderas.

// (OR) Verdadero si al menos una condición es verdadera.

! Not invierte el valor de verdad (Verdadero a falso y viceversa)

El operador coma (,)

permite encadenar varias expresiones. Se evalúan de izquierda a derecha y la expresión completa toma el valor de la última subexpresión.

Tabla de jerarquía / prioridad de operadores (de mayor a menor):

parentesis (), corchetes []

operadores unarios !, ++, --, - (negación)

multiplicativos *, /, %

aditivos +, -

Relacionales <, <=, >, >=

igualdad ==

lógicos && seguido de //

asignación =, +=, -= etc.

Questions

¿cómo se evalúan las expresiones compuestas con múltiples operadores?

¿cuál es la regla de corto-circuito en los operadores lógicos?

Summary:

Las expresiones lógicas combinan múltiples condiciones relacionadas. Comprender la jerarquía de operadores es crucial para garantizar que las expresiones se evalúen exactamente en el orden previsto por el programador.